北京理工大学
Beijing Institute of Technology

Spring 2025

操作系统

宋 红

Lecture 19: Linux存储器管理

内容:了解Linux系统的物理内存的管理、

虚拟存储空间的管理策略和方法。

1. 物理内存的管理

➢ 内存用途

- 存储内核映像
- 其它可动态加载内核模块内存需求
- 进程用户页面的需求
- 缓冲需求

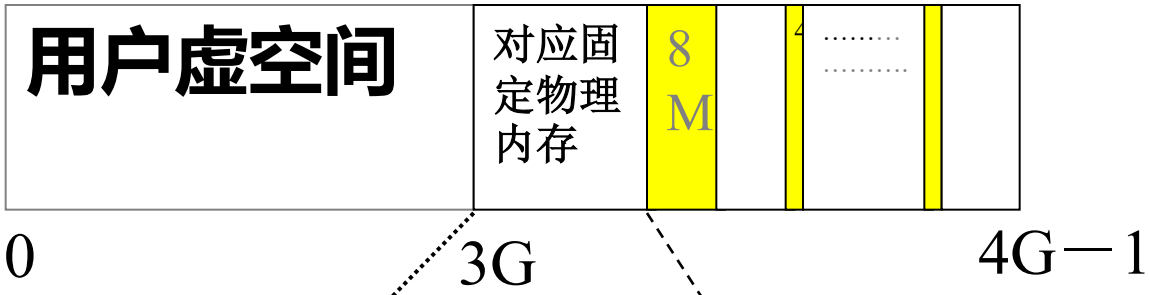
➢ 功能要求

- 快速响应请求
- 尽可能地利用内存同时减少内存碎片

➢ 解决方案

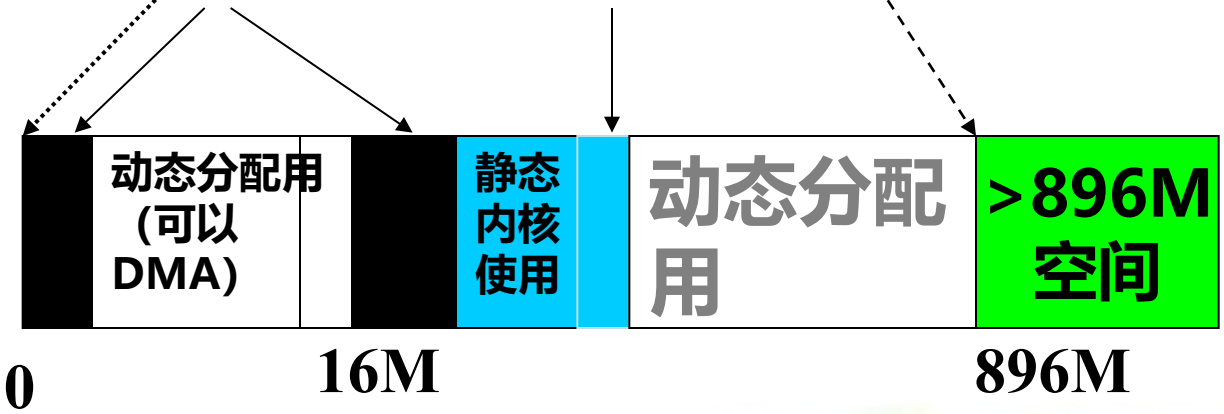
- 基于区域的 (buddy)伙伴系统及slab分配器

➢进程虚地址空间



➢物理内存

硬件使用 mem_map[]（数组大小为页帧数）



➢页帧与区域

页帧: 物理内存以页帧为基本单位，页帧的**大小**固定为**4K**。

区域： 物理内存分成**三个区域**

- **ZONE_DMA** 低于16MB的内存，是DMA方式能够访问的物理内存。一般用于I/O缓冲区;
- **ZONE_NORMAL** 介于16MB与896MB之间，直接被内核虚地址映射。部分用于内核程序及常数，部分用于动态表格或用于用户空间页;
- **ZONE_HIGHMEM** 高端内存，超过896MB以上的部分。用于存放动态映射页面。

在内存分配时要根据区域访问优先级别来决定分配次序。比如为DMA方式分配内存，DMA ZONE是唯一符合要求的区域。而通常的内存分配，则可先在ZONE_HIGHMEM内存去分配，若不能满足，再尝试 ZONE_NORMAL 进行，如果还不能满足要求，则在ZONE_DMA尝试。

➢伙伴(Buddy)算法

Linux对每个区域采用了伙伴算法来对空闲内存页帧进行管理，减少外部碎片。

当要为进程用户空间页面、系统I/O缓冲区、系统存储对象（如表格；动态模块等）分配页帧时调用伙伴算法。

BUDDY算法思想：以16个页面为例，空闲页面构成五个链表

(1)初始态

- 连续16个页面 [1-16]
- 连续8个页面
  - 连续4个页面
  - 连续2个页面
  - 连续1个页面

(2)分配两个页面,剖分[1-16]，[1-2]被使用，状态如下

- 连续16个页面
- 连续8个页面 [9-16]
  - 连续4个页面 [5-8]
  - 连续2个页面 [3-4]
  - 连续1个页面

分配方法：尽可能从合适大小的链表寻找，如有，则摘除返回页面。否则寻找更大的连续空闲页面，剖成两个大小相等伙伴，将后者插入合适位置。再看前者是否满足要求，若不，继续剖分。

(3)再申请四个页面，[1-2] [5-8]被使用

连续16个页面

连续8个页面 [9-16]

连续4个页面

连续2个页面 [3-4]

连续1个页面

(4)释放页面[1-2]，看能否合并伙伴

连续16个页面

连续8个页面 [9-16]

连续4个页面 [1-4]

连续2个页面

连续1个页面

释放过程：看能否与空闲伙伴合并，若可以，则合并成更大的连续页面并继续重复能否与空闲伙伴合并的过程。最后插入合适位置。

两组连续页面块被认为是一对"伙伴"必须满足如下条件:

（1）两个块具有相同的大小，记作b。

（2）它们的物理地址是连续的。

（3）第一块的第一个页帧的编号必须是2b的倍数。

➢伙伴算法的优点

分配页面时尽量动用小的连续页面,回收页面时则尽可能将空闲的伙伴合成大的连续页面。

➢伙伴系统的操作

- 申请空间的函数为__alloc_pages()
- 释放函数为__free_pages_ok()
- 当在申请内存发现页面短缺时还会唤醒kswapd内核线程运行,该线程会腾出一些空间以满足要求

➢slab分配器

存在问题： 伙伴系统是以页帧为基本分配单位, 对于小对象容易造成是内部碎片。

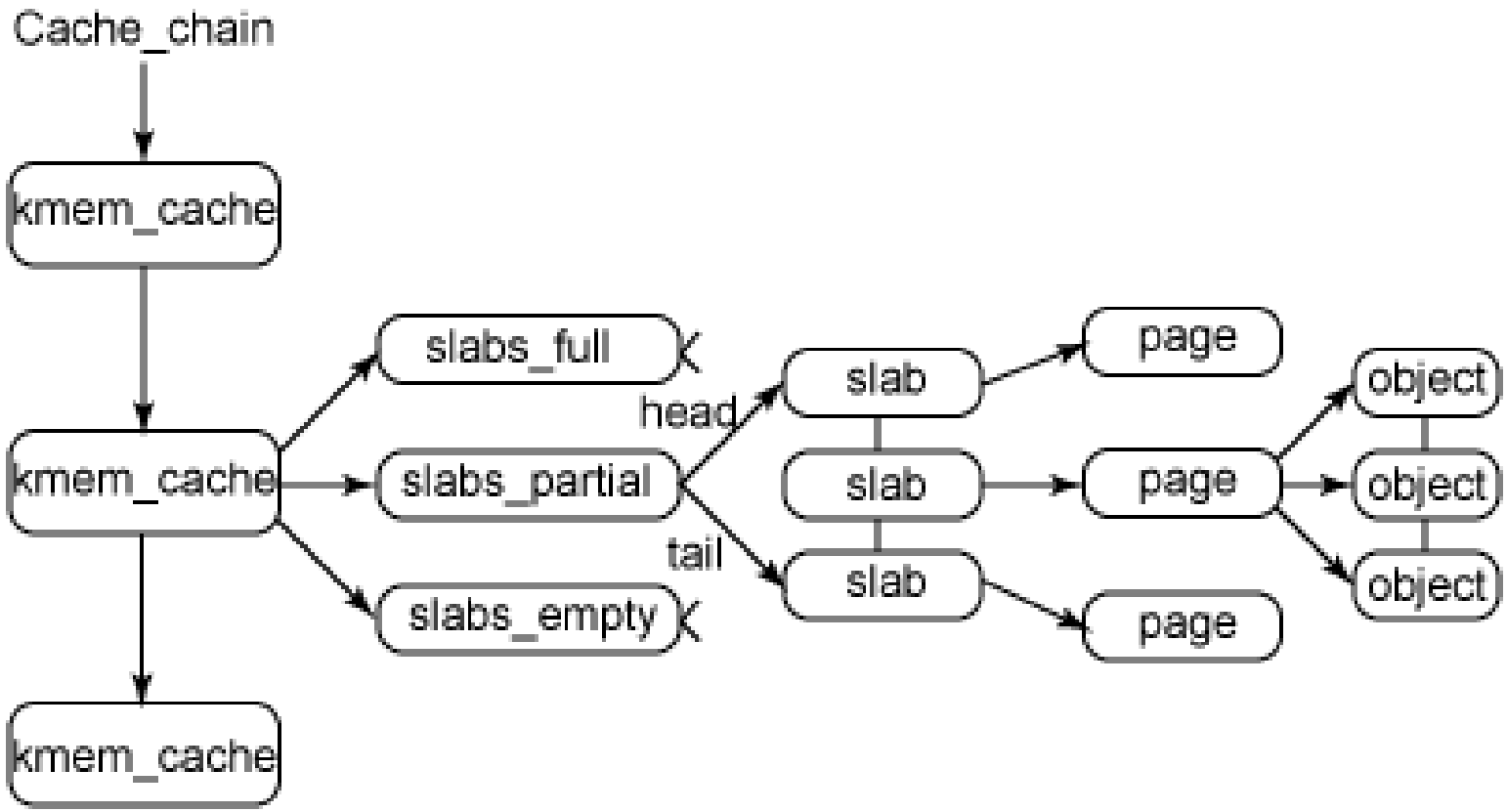
解决办法： 基于伙伴系统的slab分配器。

slab分配器的基本思想: 为经常使用的小对象建立缓冲, 小对象的申请与释放都通过slab 分配器来管理。slab 分配器再与伙伴系统打交道。

好处： 其一是充分利用了空间,减小了内部碎片。其二是管理局部化,尽可能少地与伙伴系统打交道,从而提高了效率。

➢slab分配器的构成

- 为不同的常用对象生成不同的缓冲,每个缓冲存储相同类型的对象。
- 某种对象的缓冲区由一连串的slab构成,每个slab又由一个或多个连续的物理页帧组成,包含了若干同种类型的对象。
- slab对象用结构slab_t来描述,某种特定对象建立的slab队列都有个队头kmem_cache_t
- 系统有一个slab队列头的slab队列,其对象是其它对象的队头,其队头也是一个kmem_cache_t结构,叫cache_chain。



slab 分配器的主要结构

除了上面讨论的特定对象的缓冲外，Linux还提供了十三种通用的缓冲，其存储对象的单位大小分别为32字节，64字节，128字节，256字节，512字节，1KB，2KB，4KB，8KB，16KB，32KB，64KB和128KB。这些缓冲用来满足特定对象之外的普通内存需求。单位大小级数增长保证了内部碎片率不超过50%。

➢ slab分配器的相关操作:

(1) kmem_cache_create()

该函数创建一种特定对象的kem_cache_t结构,并加入cache_chain 所管理的队列。如下代码创建了inode_cache:

```
kmem_cache_create("inode_cache",
  sizeof(structinode), 0,
  SLAB_HWCACHE_ALIGN, init_once,NULL);
```

kmem_cache_create() 函数一开始分配一个kmem_cache_t结构,然后进行一系列运算,以确定最佳的slab 构成。包括:每个slab由几个页面组成,可包含几个对象; slab的控制结构应该在slab外面集中存放还是放在每个slab的尾部。

(2)kmem_cache_alloc()与kmem_cache_free()

当需要分配一个拥有专用slab队列的对象时,应该通过kmem_cache_alloc()函数,相反的动作则是kmem_cache_free()函数。

分配的对象来自slab缓冲，释放的对象归还slab缓冲,通过某种标识表明对象是可用还是已被占用。

(3)kmem_cache_grow()与kmem_cache_reap()

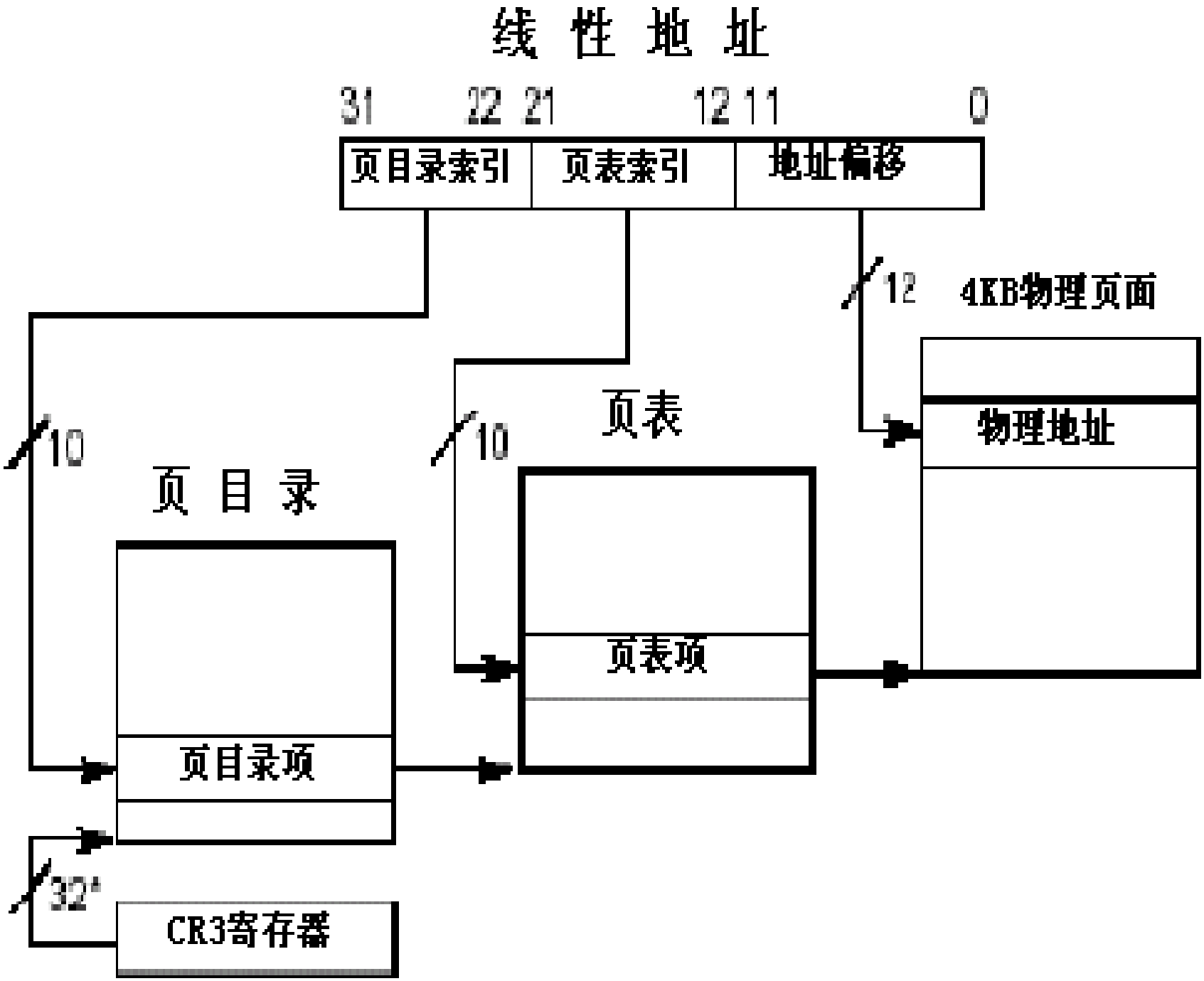
- kmem_cache_create()函数只是建立了所需的专用缓冲区队列的基础设施,所形成的slab队列是个空队列。
- 而具体slab的创建则要等需要分配缓冲区时,却发现队列中并无空闲的缓冲区可供分配时,再通过kmem_cache_grow()向伙伴系统申请空间。
- kswapd会定时调用kmem_cache_reap()来“回收”缓冲区队列的页帧。

(4)kmalloc()函数与kfree()函数

从通用的缓冲区队列中申请和释放空间。

2. 进程空间的管理

- 每个进程都有自己独立的虚拟地址空间。虚拟内存以页为基本单位,大小等同于物理页帧。
- 每个进程空间通过进程的页目录、页表实现与物理内存间的映射。
- 进程通过按需分页得到物理内存。
- 进程空间主要由mm_struct结构描述。该结构包含了进程空间的两个组成部分：进程的页目录以及指向vm_area_struct结构的指针。


线 性 地 址
31
22 21
12 11
0
页目录索引
页表索引
地址偏移
10
10
12
4KB物理页面
物理地址
页 目 录
页表
页表项
页目录项
32
CR3寄存器

➢页表机制

以X86体系结构为例。i386系列既支持分段机制，也支持分页机制，Linux主要采用了分页机制，页面可以映射到任一物理页帧。i386下进程的线性地址为32位,分为以下三个部分:

页目录段 置于高10位。记录在页目录中的索引。
页表段 占据中间的10位。记录在页表中的索引。
偏移段 占据低12位。用来表示在4KB的页帧中的偏移。

➢ 进程空间线性地址转换成为物理地址的过程：

每个进程都有一个**页目录**,当进程运行时,**寄存器CR3**指向该页目录的基址。

(1)从**CR3**取得**页目录**的基地址。页目录用一个物理页帧存储,用来保存**页表**的基址。

(2)以线性地址的页目录段为索引,在**页目录**中找到**页表**的基址。每一**页表**也是用一个物理页帧存储，用来保存物理页帧号。

(3)以线性地址的**页表**段为索引，在**页表**中找相应的物理页帧号。

(4)物理页帧号加上线性地址的偏移段即得到了对应的**物理地址**。

物理页帧是4KB对齐的，页目录项和页表项都只需要20位保存物理地址的12-31位,还有12位可以用于控制或其它目的。**几个重要的标志位是:**

存在位: 表示项对应的物理页是否存在。

读/写位: 表示是只读还是可写,起保护作用。

访问位: 表明该页是否被访问过。

脏标志位:表明该页是否被写过。

采用两级分页的好处是节省了存储空间。

Linux采用的是三级页表模型,在页目录和页表之间还有页中间目录层。对于32位机器,页中间目录层实际不起作用。

80X86处理器内部有称作TLB的专门缓冲用来加速地址转换过程。

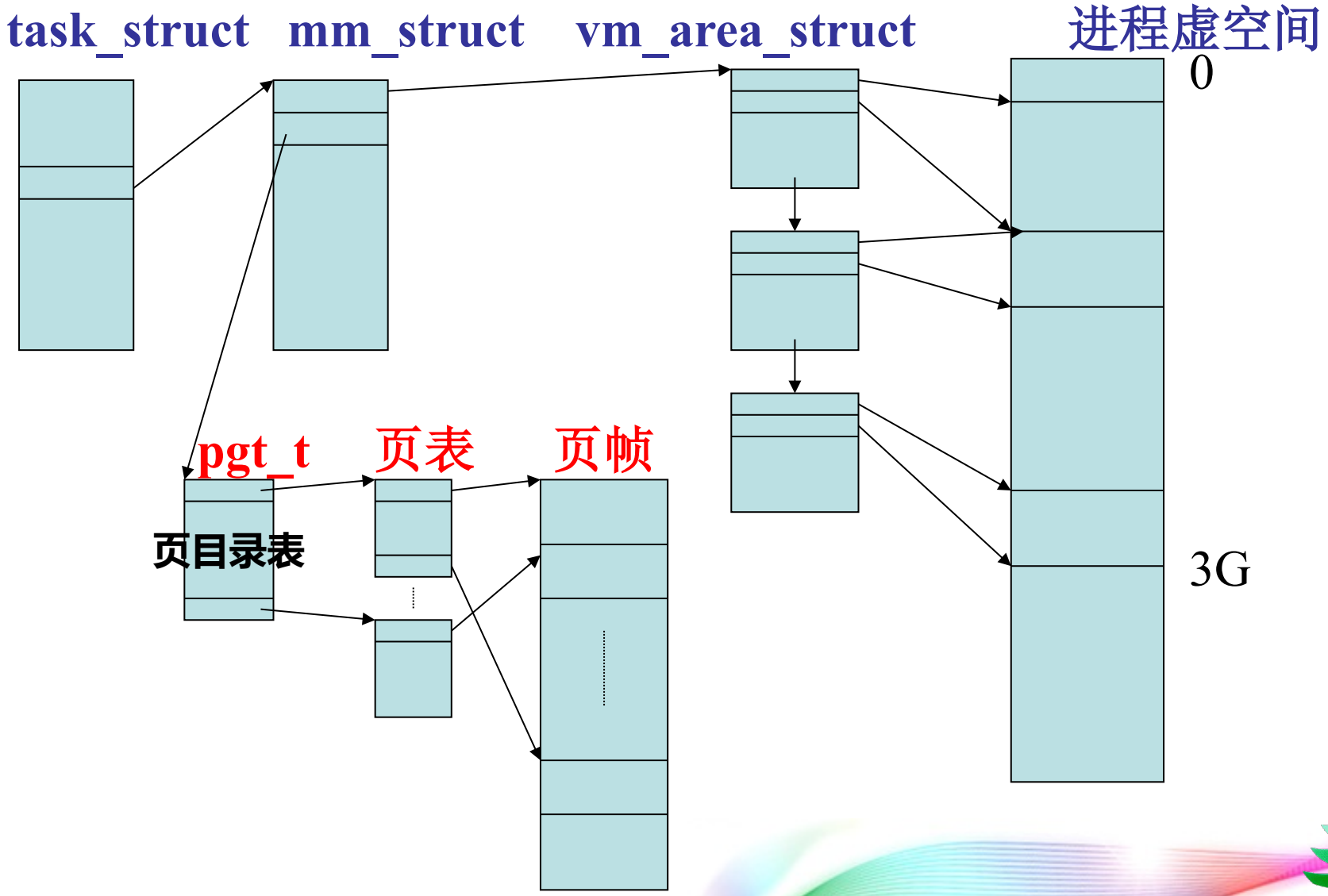
➢进程空间的管理

（1） vm_area_struct结构

- 对于32位机器,共有4G地址空间。其中前3G可以为进程直接访问,称为用户空间，而后1G空间为内核所用，称为内核空间。

- 目前绝大多数程序都不会使用全部的3G空间。组成进程空间的映象由多个区间, Linux用一系列的vm_area_struct 结构描述不同的区间。

- 进程页面必定属于某个vm_area_struct结构,但是属于vm_area_struct结构的页面不一定在页表中有数据项 。



➢进程空间的相关系统调用

- exec调用会抛弃原来的地址空间并根据可执行文件内容生成相应的新的地址空间。
- fork系统调用，通常情况下：

（1）子进程会拷贝一份父进程的地址空间，包括父进程的vm_area_struct结构以及页目录、页表均会被拷贝。

(2) 写时拷贝(COPY ON WRITE)

- 父进程页表项所标识的有效物理页帧并不会立即拷贝,而是为子进程和父进程共享,即父进程和子进程的同一页表项指向同一个物理页帧,并且页表项的相应标志位均标记位“只读”,
- 写入该页帧时就会引发页面异常进一步处理。

好处是:(1)节省了开销,

(2)“懒惰策略",分散式复制的效果相比起一次性集中复制即时响应效果更好。

- 系统调用brk()用来改变堆的大小,增大进程空间。
- exit()结束进程并销毁进程空间。
- mmap()创建文件的内存映象,扩展进程空间,而munmap()行为与之相反。
- shmat()创建一个共享内存区而shmdt()则释放共享内存区.

➢ 页面异常的处理

导致页面异常的原因有:

(1)编程错误，可分为内核程序错误和用户程序错误。

(2)操作系统故意引发的异常，操作系统合理利用硬件机制在适当时间触发异常,使得该异常的处理程序被调用以达到预期目的。如缺页异常。

页面异常的处理程序是do_page_fault()函数，该函数有两个参数：

- 一个是指针,指向异常发生时寄存器值存放的地址。
- 另一个错误码,由三位二进制信息组成

第0位　访问的物理页帧是否存在

第1位　写错误还是读错误或执行错误

第2位　程序运行在核心态还是用户态

do_page_fault()函数的执行过程如下：

(1)首先得到导致异常发生的线性地址,对于X86该地址放在CR2寄存器中。

(2)检查异常是否发生在中断或内核线程中,如是,则进行出错处理。

(3)检查该线性地址属于进程的某个vm_area_struct区间。如果不属于任何一个区间,则需要进一步检查该地址是否属于栈的合理可扩展区间。一但是用户态产生异常的线性地址正好位于栈区间的vm_start前面的合理位置,则调用expand_stack()函数扩展该区间,通常是扩充一个页面,但此时还未分配物理页帧。

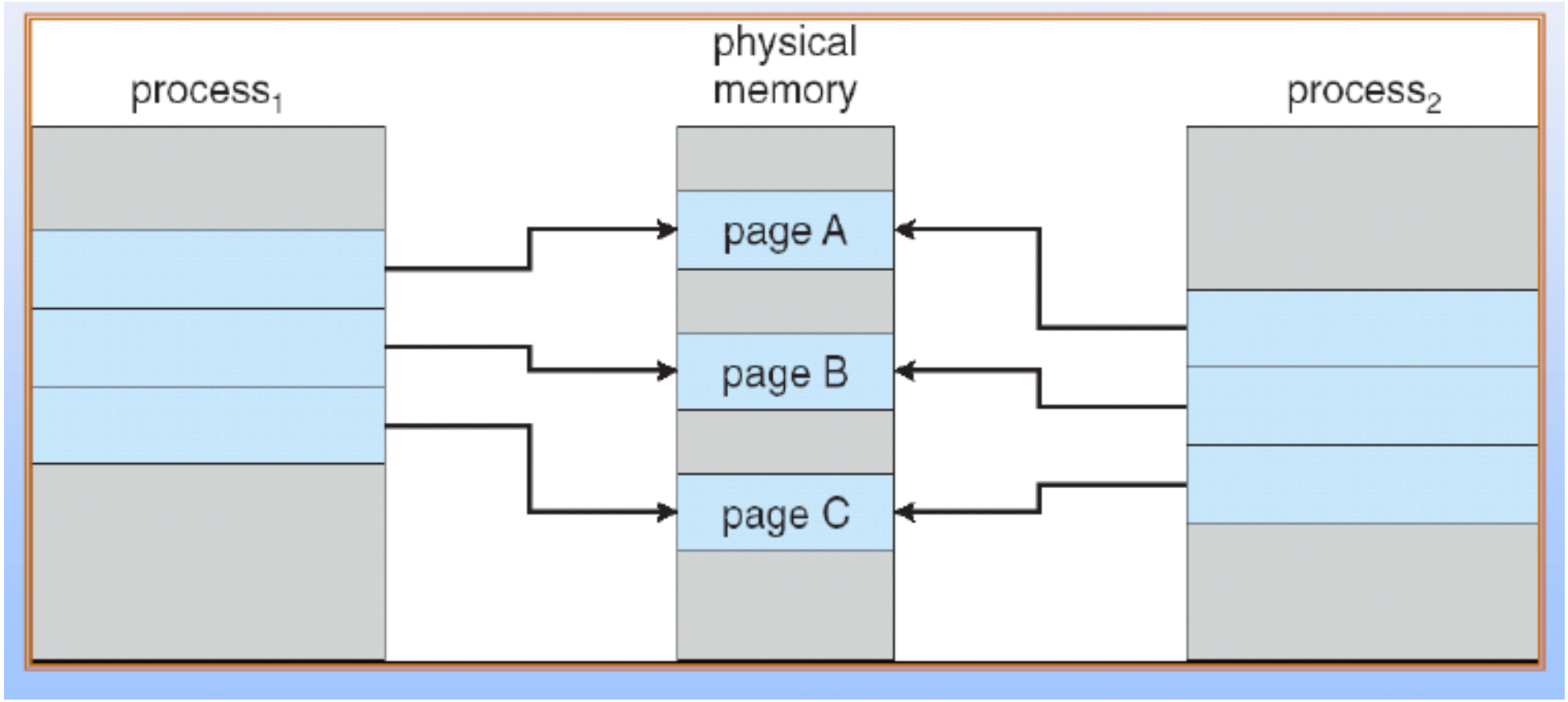
至此,线性地址必属于某个区间。

(4)根据错误码的值确定下一个步骤:

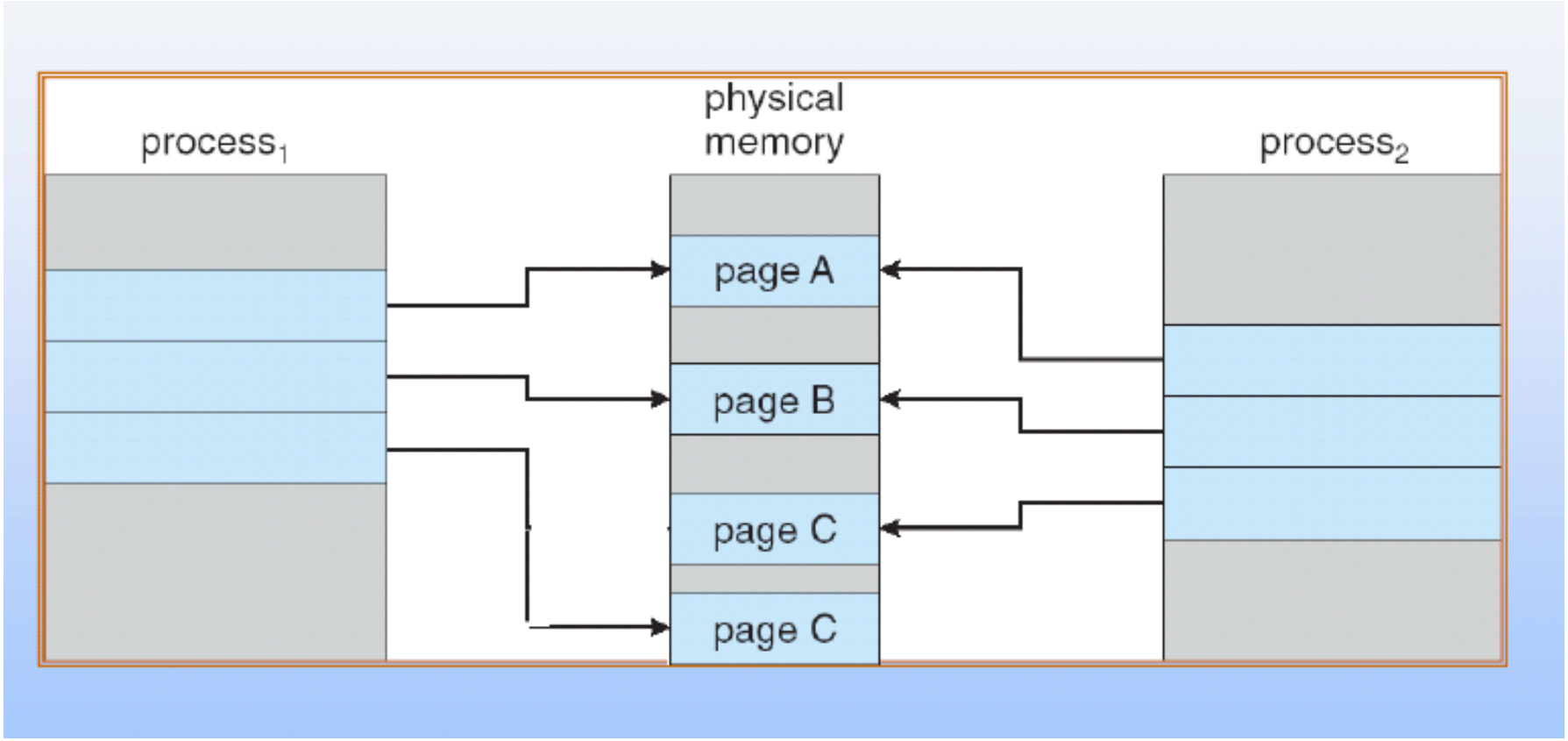
如果错误码的值表示为写错误,则检查该区间是否允许写,不允许则进行出错处理。如果允许就是属于前面提到的写时拷贝。如果错误码的值表示为页面不存在,这就是所谓的按需分页(demand paging)。

写时拷贝的处理过程：

- 首先改写对应页表项的访问标志位表明其刚被访问过,这样在页面调度时该页面就不会被优先考虑。
- 如果该页帧目前只为一个进程单独使用,则只须把页表项置为可写。
- 如果该页帧为多个进程共享,则申请一个新的物理页面并标记为可写,复制原来物理页面的内容,更改当前进程相应的页表项,同时原来的物理页帧的共享计数减一。



进程1修改页C之前


process$_1$
physical memory
process$_2$
page A
page B
page C
page C

进程1修改页C之后

按需分页的处理过程:

- 第一种情况页面从未被进程访问,这种情况页表项的值全为0。

  (a) 如果所属区间的vm_ops->nopage不为空,表示该区间映射到一个文件并且vm_ops->nopage指向实现页面装入的函数,此时调用该函数装入该页面。

  (b) 如果vm_ops或vm_ops->nopage为空,则调用do_anonymous_page()申请一个页帧;

- 另一种情况是该页面被进程访问过,但是目前已被写到交换分区, 页表项的存在标志位为0,但其它位被用来记录该页面在交换分区中的信息。调用do_swap_page()函数从交换分区调入该页面。